

EEE 402 (January 2025)

Artificial Intelligence and Machine Learning Laboratory

Final Project Report

Section: G2 Group: 02

Stock Market Prediction with Sentiment Analysis

Course Instructors:

Dr. Mohammad Ariful Haque, Professor
Akif Hamid, Part-Time Lecturer

Signature of Instructor: _____

Academic Honesty Statement:

IMPORTANT! Please carefully read and sign the Academic Honesty Statement, below. Type the student ID and name, and put your signature. You will not receive credit for this project experiment unless this statement is signed in the presence of your lab instructor.

"In signing this statement, We hereby certify that the work on this project is our own and that we have not copied the work of any other students (past or present), and cited all relevant sources while completing this project. We understand that if we fail to honor this agreement, We will each receive a score of ZERO for this project and be subject to failure of this course."

Signature: _____ Full Name: Md. Rifat Rahman Student ID: 2006137	Signature: _____ Full Name: Md. Shihab Sheikh Student ID:2006176
Signature: _____ Full Name: Orjun Kumar Paul Student ID:2006186	Signature: _____ Full Name: Jain Uddin Ahmad Student ID:2006187
Signature: _____ Full Name: Mushfikur Rahman Navin Student ID:2006190	Signature: _____ Full Name:A.M Mostain Ahmed Sayami Student ID:2006193

Table of Contents

Dr. Mohammad Ariful Haque, Professor.....	1
1 Abstract	1
2 Introduction.....	1
3 Design	1
3.1 Problem Formulation (PO(b)).....	1
3.1.1 Identification of Scope.....	2
3.1.2 Literature Review.....	2
3.1.3 Formulation of Problem.....	3
3.1.4 Analysis	4
3.2 Design Method (PO(a)).....	4
I. Dataset collection:	5
II. Data pre-processing and feature extraction:	5
III. Implementation of VADER model:	5
4 Implementation:	10
5 Design Analysis and Evaluation	12
5.1 Design Considerations (PO(c)).....	12
5.1.1 Results and Analysis.....	14
5.1.2 Discussions:	19
5.2 Limitations:.....	20
6 Reflection on Individual and Team work (PO(i)).....	21
6.1 Individual Contribution of Each Member :.....	21
Individual Contribution.....	21
6.2 Log Book of Project Implementation	22
7 Communication to External Stakeholders (PO(j)).....	24
7.1 Executive Summary.....	24
7.2 Github Link: https://github.com/navin2006190/Stock_market_prediction_with_sentiment_analysis.git	24
8 Future Work (PO(l)):	24
9 References:.....	25

1 Abstract

Stock market prices are always changing and can be very hard to predict. In this project, we try to improve stock price prediction by using both past stock data and public sentiment which we have been collected from social media news. First, we use a sentiment analysis tool called **TextBlob** to find out whether news headlines or social media posts about the stock market are positive, negative, or neutral. This gives us a **sentiment score**. Then we use this score as one of the inputs in a machine learning model called **LSTM** (Long Short-Term Memory), which is good at working with data that changes over time. By combining sentiment with past stock prices, our model tries to make better predictions about future stock prices. This approach shows that public opinion can be helpful in forecasting stock market movements.

2 Introduction

The stock market is influenced by many factors such as company performance, global events, and most importantly, public sentiment. In recent years, people have started using social media and news websites to share their opinions about companies and the economy. These opinions can have a big impact on stock prices. For example, if many people are saying positive things about a company, its stock price might go up. On the other hand, negative news or posts can cause prices to drop.

In this project, we aim to predict stock market prices by combining two things: historical stock data and public sentiment. First, we analyze the sentiment of news headlines or social media texts using a tool called **VADER**, which is specially made to understand the tone (positive, negative, or neutral) of short texts. This gives us a **sentiment score** that tells us how people are feeling about the market.

After that, we use this sentiment score along with previous stock prices in a deep learning model called **LSTM** (Long Short-Term Memory). LSTM is a type of Recurrent Neural Network (RNN) that works well with time-series data, like stock prices over days or weeks. By combining both numeric data and public opinion, we try to build a model that can better understand and predict how stock prices might change in the future.

This project shows how machine learning and natural language processing (NLP) can work together to create smarter tools for stock market analysis

3 Design

3.1 Problem Formulation (PO(b))

Forecasting stock market prices is a complicated endeavor as prices are influenced by numerous factors, such as financial information, worldwide occurrences, and collective sentiment. Conventional models primarily rely on historical stock prices and technical indicators for making forecasts. Nonetheless, they frequently overlook the impact that news and social media sentiment can have on market fluctuations.

This project focuses on enhancing stock price forecasting by incorporating sentiment

analysis as an extra feature. Our primary objective is to develop a model capable of forecasting future stock prices by utilizing historical stock information and public sentiment derived from texts like news headlines or tweets.

We divide the issue into two components:

Sentiment Evaluation: Employ the VADER model to examine the text data and produce a sentiment score (positive, negative, or neutral).

Stock Price Forecasting: Employ an LSTM-driven deep learning model that utilizes historical stock prices along with sentiment scores to anticipate the future stock price.

By merging these two components, we aim to create a model that is more precise and attuned to actual events and perspectives

3.1.1 Identification of Scope

The scope of this project is to build a RNN model (LSTM model) to predict the closing price of stock market of a company. Before that a sentiment analysis has been done using a built in VADER model based on the news of different social medias. The VADER model gives a sentiment score which is then used as a feature of predicting stock market closing price.

3.1.2 Literature Review

Predicting stock market prices has always been a popular topic in both financial and academic research. Many traditional models like **ARIMA(Auto Regressive Integrated Moving Average)**, **Linear Regression**, and **Support Vector Machines (SVM)** have been used to forecast stock prices based on past numerical data. However, these models often fail to capture the emotional and psychological factors that influence market movements.

In recent years, researchers have started exploring the use of **sentiment analysis**—a branch of Natural Language Processing (NLP)—to understand how public opinion affects stock prices. Sentiment analysis involves analyzing texts like news articles, tweets, or financial blogs to determine whether the content is positive, negative, or neutral.

One widely used tool for sentiment analysis is the **TextBlob** model, which is especially effective for analyzing short texts and social media posts. According to Hutto and Gilbert (2014), VADER combines a dictionary of sentiment-related words with rules that consider punctuation, capitalization, and degree modifiers. It has been used in several financial prediction studies due to its simplicity and good performance in real-time applications.

For time-series prediction, especially in financial data, **Recurrent Neural Networks (RNNs)** and more specifically, **Long Short-Term Memory (LSTM)** networks have shown promising results. LSTM was introduced by Hochreiter and Schmidhuber (1997) to solve the problem of learning long-term dependencies in sequence data. It has memory cells and gates that allow it to remember important information for a longer period, which is useful in stock prediction where past trends influence future prices.

Many researchers have combined sentiment analysis with LSTM models to improve prediction accuracy. For example:

- **Zhang et al. (2018)** showed that adding Twitter sentiment scores to LSTM models improved the prediction of stock trends for tech companies.
- **Chen and Zhao (2019)** used financial news sentiment with LSTM and found better accuracy compared to using historical data alone.
- **Kumar and Ravi (2020)** conducted a survey and concluded that hybrid models combining machine learning with textual sentiment generally outperform single-source models.

These studies show that combining **quantitative data (like historical prices)** with **qualitative data (like sentiment scores)** can lead to more reliable and realistic stock market predictions.

Despite the progress, challenges still exist, such as:

- Noise and fake information in social media.
- Time-alignment of sentiment with stock market events.
- Market unpredictability due to sudden global events (e.g., COVID-19, wars, etc.).

In this project, we build upon these ideas by using **VADER** for sentiment analysis and **LSTM** for time-series prediction, aiming to design a model that combines the strength of both methods to forecast stock prices more effectively.

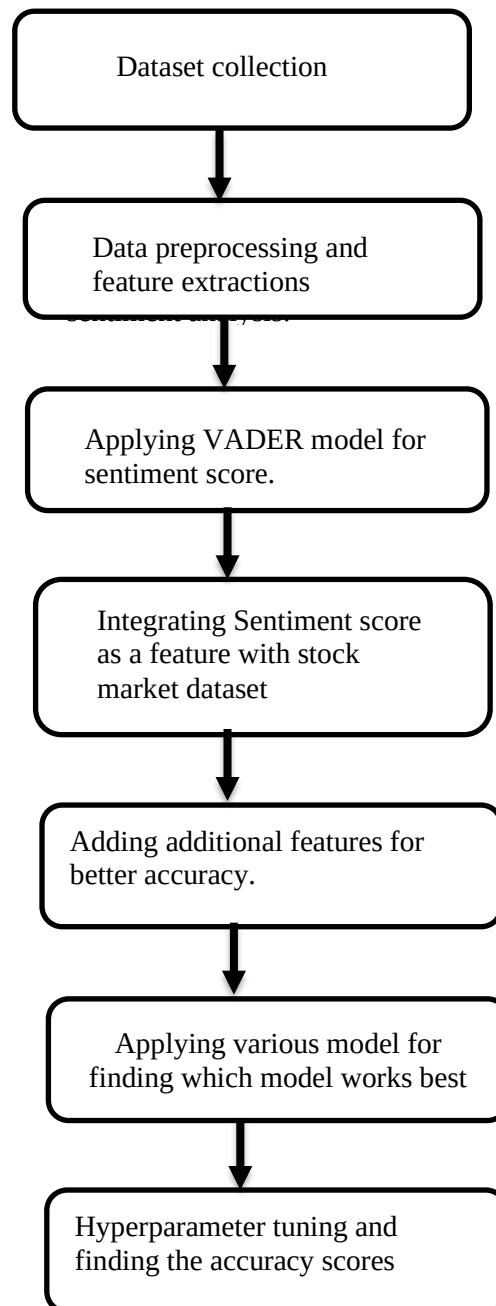
3.1.3 Formulation of Problem

- I. The main challenges we have faced during this problem is to manage the correlated datasets both for sentiment analysis and stock market price prediction. First we were not getting correlated datasets so we first built a model using uncorrelated datasets and we observed our model is performing well. Then we have found two correlated datasets, which have both connection between sentiment analysis datasets and stock market prize prediction dataset.
- II. After finding correlated datasets, we have faced another problem and it was about time. The dataset we have got was from the news and stock market's price from 2008 to 2016 which is very old and we may not use this model properly to predict the stock market price for current time.

3.1.4 Analysis

3.2 Design Method (PO(a))

Here, now we will discuss the design method of our project step by step. At first, we will provide a flowchart of our design procedure then we will discuss the whole method in details.



I. Dataset collection:

The datasets for sentiment analysis and stock market price prediction can be collected from Kaggle datasets or many renowned research papers.

II. Data pre-processing and feature extraction:

The collected dataset for sentiment analysis is needed to clean properly to get better sentiment score. Otherwise, the sentiment score will be uncorrelated and the accuracy of the model will be poor. So, first, it is required to preprocess text data before apply them into the TextBlob model.

III. Implementation of VADER model:

VADER (Valence Aware Dictionary and Sentiment Reasoner) is a rule-based sentiment analysis model designed specifically to analyze sentiment in social media texts, news headlines, and short, informal texts.

- Created by C.J. Hutto and Eric Gilbert in 2014.
- It's included in Python's nltk library.
- Outputs a sentiment score: how positive, negative, or neutral the text is.

VADER combines two main things:

1.A Sentiment Lexicon (Dictionary)

- It has a list of English words, each assigned a valence score between -4 (extremely negative) and +4 (extremely positive).
- Examples:
 - “good” = +1.9
 - “great” = +3.1
 - “bad” = -2.5
 - “hate” = -3.2

2. Rule-Based Heuristics

VADER doesn't just look at the words. It also follows a set of rules to adjust the senti+ment score based on how the sentence is written

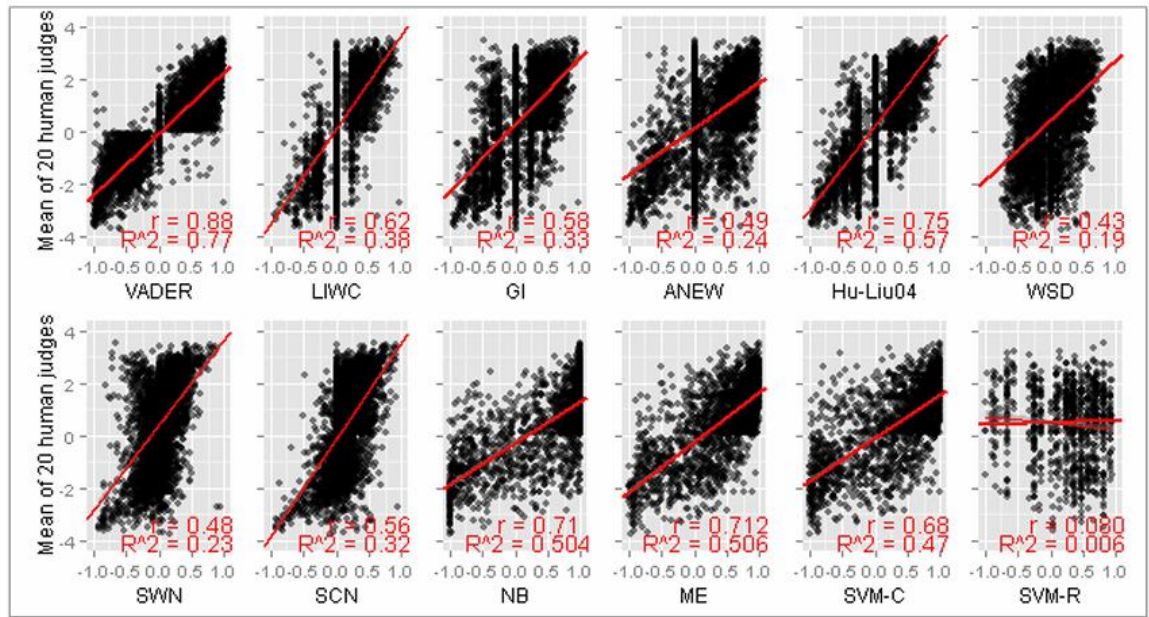


Figure 3: Sentiment scores from VADER and 11 other highly regarded sentiment analysis tools/techniques on a corpus of over 4K tweets. Although this figure specifically portrays correlation, it also helps to visually depict (and contrast) VADER's classification precision, recall, and F1 accuracy within this domain (see Table 4). Each subplot can be roughly considered as having four quadrants: true negatives (lower left), true positives (upper right), false negatives (upper left), and false positives (lower right).

IV. Integrating sentiment score as a feature with stock market dataset:

Now, the important thing is to do the integration of sentiment score with the stock market dataset, which will predict the closing price of the stock. The sentiment score data will be treated as a valuable feature for stock market price prediction. Now, in the dataset for sentiment analysis there may be repeated news for the same day, but in the stock price dataset there is only a single entry for each day. For this, to match with the row number, the sentiment score may be averaged to get a single value for each day.

V. Adding some additional features to get better accuracy:

In the stock market price prediction dataset, there are features like opening price, maximum price, minimum price, adjusted price, and sentiment score. If time series analysis-based RNN models like LSTM or GRU are used, then a problem might arise of data overfitting. To avoid this problem, a time series is needed to introduce with the model where the output of the next day depends on the inputs of some previous days. So, that's why to increase the accuracy of the model, some additional features are badly required to add with the existing features which will work on time series analysis to avoid overfitting. Some of them are:

MA7 and **MA20** refer to 7-day and 20-day moving averages, which show the average closing price over those periods. They help smooth short-term noise and highlight the stock's overall trend, with MA7 showing short-term trends and MA20 showing mid-term movement.

MACD (Moving Average Convergence Divergence) is a momentum indicator that subtracts the 26-day EMA from the 12-day EMA. It helps identify trend changes, and its signal line (9-day EMA of MACD) is used to trigger potential buy or sell signals.

The **20-day Standard Deviation (20SD)** measures how much the price varies from its average. A high standard deviation means high volatility, while a low value indicates stable price behavior.

Upper and Lower Bollinger Bands are based on the 20-day moving average plus or minus two times the 20SD. They form a price envelope to detect overbought or oversold conditions when the price touches these bands.

The **EMA (Exponential Moving Average)** is similar to a simple moving average but gives more importance to recent prices. It reacts faster to recent changes, making it useful for short-term trend detection.

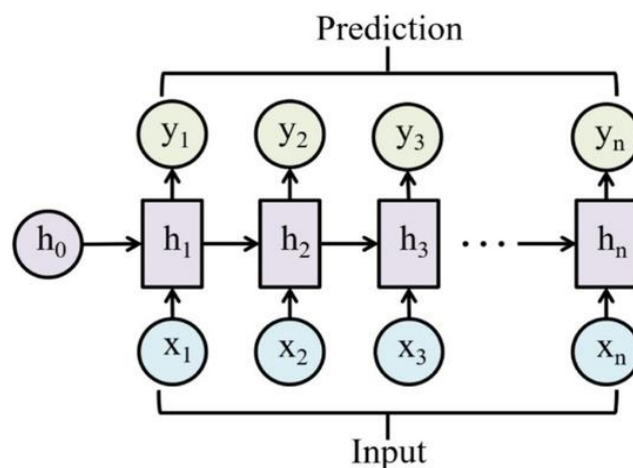
RSI-14 (Relative Strength Index) measures the strength of recent price movements over 14 days. It ranges from 0 to 100, where values above 70 suggest an overbought market and below 30 indicate oversold conditions.

SMA-14 is a simple average of the past 14 closing prices. It is commonly used alongside RSI to confirm trends and reversals.

Log Momentum captures the logarithmic change in price over time. It helps the model understand the speed and direction of price movements in a scale-independent way.

VI. Applying various model for finding which model works best:

Time series analysis based problem mainly works better in RNN type models like LSTM and GRU. In ANN or CNN, they do not perform well, so it can be seen using various models by comparing the values of MSE or R2 score.



Recurrent Neural Networks (RNNs)

Recurrent Neural Networks (RNNs) are a type of neural network designed to handle sequential data such as time series, text, or audio. Unlike traditional feedforward neural networks, RNNs have a “memory” — they retain information from previous steps and use it to influence the current output. This makes RNNs well-suited for problems where the order of data matters, such as stock market prediction.

An RNN takes an input at each time step (like stock price or sentiment score on day t), processes it, and passes the result to the next time step along with its internal hidden state. This hidden state helps the network remember useful patterns over time.

The process looks like this:

- I. Input at time t : x_t
- II. Hidden state from previous time: h_{t-1}
- III. New hidden state: $h_t = f(Wx_t + Uh_{t-1} + b)$
- IV. Output: based on h_t

In practice, basic RNNs have a problem called the vanishing gradient — they forget long-term dependencies. That means they cannot remember patterns from far back in the sequence. This is a big issue for tasks like predicting stock prices, where long-term trends may matter.

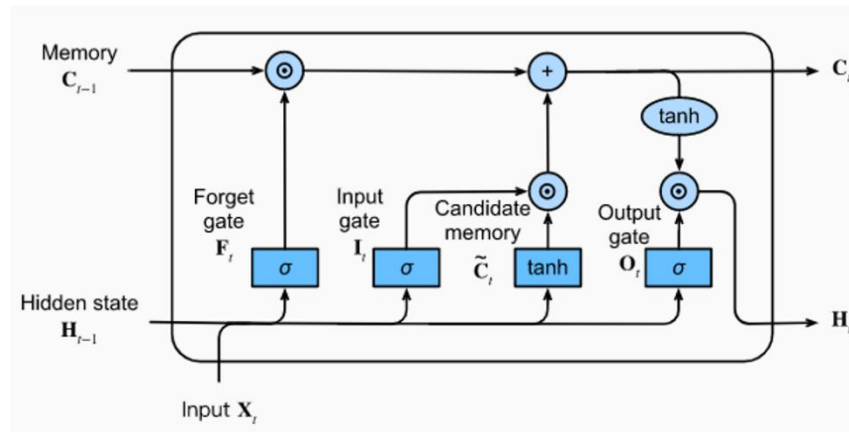
LSTM (Long Short-Term Memory)

LSTM is a special type of RNN that solves the forgetting problem by adding a cell state and gates that control the flow of information. It can remember information for long periods, which makes it very effective in time-series prediction tasks like stock prices.

Each LSTM unit has three gates:

- I. Forget Gate: Decides what to throw away from previous memory.
- II. Input Gate: Decides what new information to store.
- III. Output Gate: Decides what to output from the current cell.

This structure allows LSTM to keep only important information and ignore noise or unnecessary data from earlier time steps.



GRU (Gated Recurrent Unit)

GRU is another advanced version of RNN similar to LSTM but with a simpler structure. It combines the forget and input gates into a single update gate, making it faster to train and requiring fewer parameters.

GRU uses two gates:

- I. Update Gate: Controls how much of the past information to keep.
- II. Reset Gate: Decides how to combine new input with past memory.

Because of its simpler design, GRU often performs just as well as LSTM but with faster computation, which is useful when working with large datasets or real-time prediction.

In our project, we are dealing with stock market time-series data and sentiment scores that change over time. To capture the temporal patterns, we use an LSTM-based model. LSTM helps the model remember useful trends, market behavior, and how sentiment impacts prices across several days. GRU could also be used as an alternative due to its simplicity and comparable performance.

VII. Hyper parameter tuning and accuracy score:

In deep learning, **hyperparameters** are the settings we choose **before training the model**, such as the number of layers, learning rate, or batch size. Unlike weights and biases, which the model learns during training, hyperparameters must be selected manually or through experimentation.

For LSTM and GRU models, tuning hyperparameters is especially important to improve **prediction accuracy** and **training efficiency**.

4 Implementation:

Here, we will provide a detailed information about the implementation of our project based on the design procedure discussed in the previous section.

- I. **Dataset collection:** We collect data for sentiment analysis from built in dataset of Kaggle. Our selected dataset is mainly gives sentiment on the stock market of tech related companies like google, amazon, Microsoft etc. for a time period from 2008 to 2016.

Date	# Label	Top1	Top2	Top3	Top4
2008-08-08	0	b'Georgia 'downs two Russian warplanes' as countries move to brink of war"	b'BREAKING: Musharraf to be impeached.'	b'Russia Today: Columns of troops roll into South Ossetia; footage from fighting (YouTube)'	b'Russian tanks are moving towards the capital of South Ossetia, which has reportedly been completel...
2008-08-11	1	b'Why wont America and Nato help us? If they wont help us now, why did we help them in Iraq?'	b'Bush puts foot down on Georgian conflict'	b'Jewish Georgian minister: Thanks to Israeli training, we're fending off Russia "	b'Georgian army flees in disarray as Russians advance - Gori abandoned to Russia without a shot fire...
2008-08-12	0	b'Remember that adorable 9-year-old who sang at the opening ceremonies? That was fake, too.'	b'Russia 'ends Georgia operation'"	b'"If we had no sexual harassment we would have no children..."'	b'Al-Qa'eda is losing support in Iraq because of a brutal crackdown on activities it regards as un-I...

Then for stock market price prediction we collect data from the built-in library of python 'yfinance'. We have collected data for tech company google which contains opening, closing ,high and low price of stock market of the time period exactly same with the time period of sentiment analysis dataset .The dataset of stock market price prediction looks like below:

	Date	Close	High	Low	Open	Volume
0	2008-06-09	13.812565	14.112898	13.503815	14.064865	212324874
1	2008-06-10	13.720956	13.836088	13.537983	13.606814	146844353
2	2008-06-11	13.498861	13.799442	13.480540	13.772207	153087667
3	2008-06-12	13.690750	13.815785	13.540459	13.587008	220487355
4	2008-06-13	14.150284	14.254026	13.898480	13.902193	248303226

- II. **Sentiment score determination:** Our next task was to convert the text news headlines into sentiment score. This Python code performs sentiment analysis on news headlines from a raw RedditNews.csv file using the TextBlob library. Since the dataset is poorly formatted (with multiple headlines and dates on the same line), the code first reads the file line by line and uses regular expressions to extract pairs of dates and headlines. These pairs are stored in a structured format using a pandas DataFrame. The headlines are then cleaned to remove unwanted characters and formatting issues, such as backslashes and byte-string indicators. After cleaning, each headline's sentiment polarity score is calculated using TextBlob, which gives a value between -1 (most negative) and +1 (most positive). The sentiment scores are then grouped by date, and the average sentiment for each day is calculated to produce a daily sentiment trend. The final DataFrame, containing dates and corresponding average sentiment scores, is sorted and saved to a new CSV file Description . The output of this problem:

	Date	Close	High	Low	Open	Volume	Daily_Sentiment
0	2008-06-09	13.812565	14.112898	13.503815	14.064865	212324874	0.026156
1	2008-06-10	13.720956	13.836088	13.537983	13.606814	146844353	-0.029106
2	2008-06-11	13.498861	13.799442	13.480540	13.772207	153087667	-0.014500
3	2008-06-12	13.690750	13.815785	13.540459	13.587008	220487355	0.036611
4	2008-06-13	14.150284	14.254026	13.898480	13.902193	248303226	0.069818

- III. **Feature extraction for better accuracy:** As the problem is mainly about the forecasting based, so RNN model will perform here better. RNN like LSTM and GRU will be used here. And these models work best if the output depends on the previous some values of input. So, we will define some input length to predict the output and we will use them as features. The features we will use in this case is MA7, MA20, EMA, RSI-14, SMA-14, 20 SD etc. and check the accuracy score of our model.

IV. Models that are used for predicting the stock market price:

- Long Short Term Memory (LSTM):** We use LSTM for different time for different features to see the differences between the accuracies each time. We first run for only primary features, then sentiment score with primary features, then run for the additional features discussed above and finally all the extracted features.
- Random Forrest:** We apply this model on all extracted features with primary features.
- Linear regressor model, XgBoost regressor, ANN model:** We also use all of the extracted features for these features also.
- GRU model:** We then use another RNN based model GRU model to check the increment of accuracy score.

- V. **Hyper parameter tuning:** We tuned hyper parameter to find out the best model and used call backs like early stopping to prevent overfitting on GRU model.

5 Design Analysis and Evaluation

5.1 Design Considerations (PO(c))

In this project, we aimed to build a stock market prediction system by combining financial news sentiment analysis with machine learning (LSTM). While planning and building the project, several important design decisions were considered to ensure the model works accurately and efficiently.

1. Choice of Data Sources

The first design step was selecting the right type of dataset. We used two main data types:

- I. Financial news headlines (from RedditNews.csv) to capture public sentiment.
- II. Stock market price data (such as daily closing prices) for the company/index we want to predict.

This combination was chosen because news influences the market, and including sentiment information may help improve price prediction.

2. Sentiment Analysis Model Selection

For analyzing the sentiment of news headlines, we chose the TextBlob model (earlier VADER was also considered). This decision was based on:

- I. Easy to use and suitable for short texts like headlines.
- II. Gives a polarity score between -1 (negative) to +1 (positive).
- III. Doesn't require any training and works well on general English.

The sentiment score for each headline was calculated and then averaged per day, so we could use daily sentiment as a numerical feature.

3. Data Preprocessing Strategy

Since the original news file was not in clean CSV format, we decided to read it as plain text. Then we used regular expressions to extract the date and headline pairs properly. This was necessary because any mismatch or wrongly extracted data could reduce model accuracy.

After extraction, we also cleaned the text by removing unnecessary characters like `b' . . . '`, `\\n`, and `\\r`. Only meaningful headlines were kept for sentiment scoring.

4. Handling Time Series Data

Stock prices are time-dependent, so we needed a model that can understand the order of data. A simple regression model like Linear Regression would not be enough. That's why we chose Recurrent Neural Networks (RNN), specifically LSTM (Long Short-Term Memory), which is designed to work with sequential data.

LSTM can remember previous inputs, which helps in predicting future stock prices based on past trends and sentiments.

5. Feature Engineering

Besides sentiment, we also included technical indicators such as:

- I. Moving Averages (MA7, MA20)
- II. Relative Strength Index (RSI-14)
- III. MACD, Standard Deviation, Bollinger Bands, etc.

These features were selected because they provide trend, momentum, and volatility information about stock prices. Combining them with sentiment scores gives the model both technical and emotional signals from the market.

6. Model Architecture and Hyperparameters

We designed the LSTM model with:

- I. Input layer to accept all features.
- II. One or two LSTM layers with dropout to prevent overfitting.
- III. A Dense output layer to predict the next price.

We experimented with hyperparameters like:

- I. Number of LSTM units
- II. Batch size
- III. Learning rate
- IV. Epochs

Early stopping and validation checks were used to prevent overtraining.

7. Evaluation Method

To measure model performance, we used metrics like:

- Mean Squared Error (MSE) for numeric accuracy.
- Root Mean Squared Error (RMSE) to compare predicted vs. actual prices.

We also plotted the actual and predicted stock prices to visually verify how close the predictions were to the real market data.

5.1.1 Results and Analysis

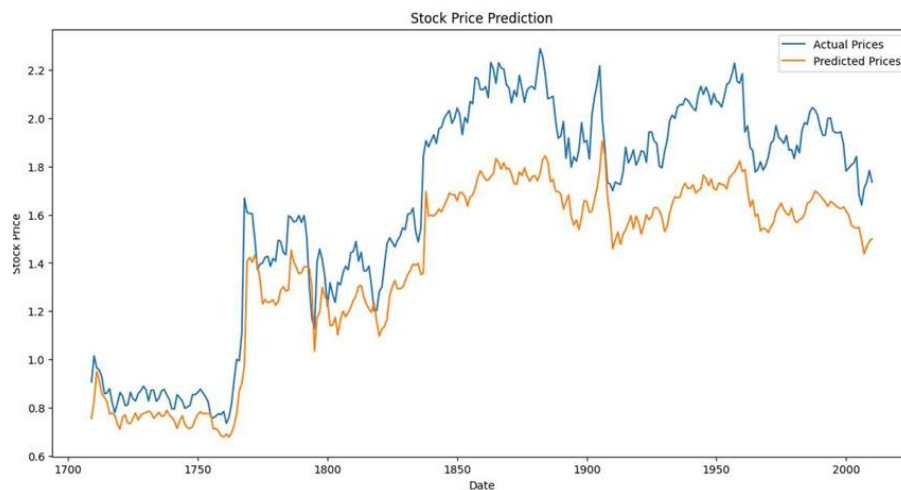
Here we will discuss about the accuracy score and graphs of predicted and actual output for different models we discussed earlier.

a. LSTM model with primary features only:

Model: "sequential_18"

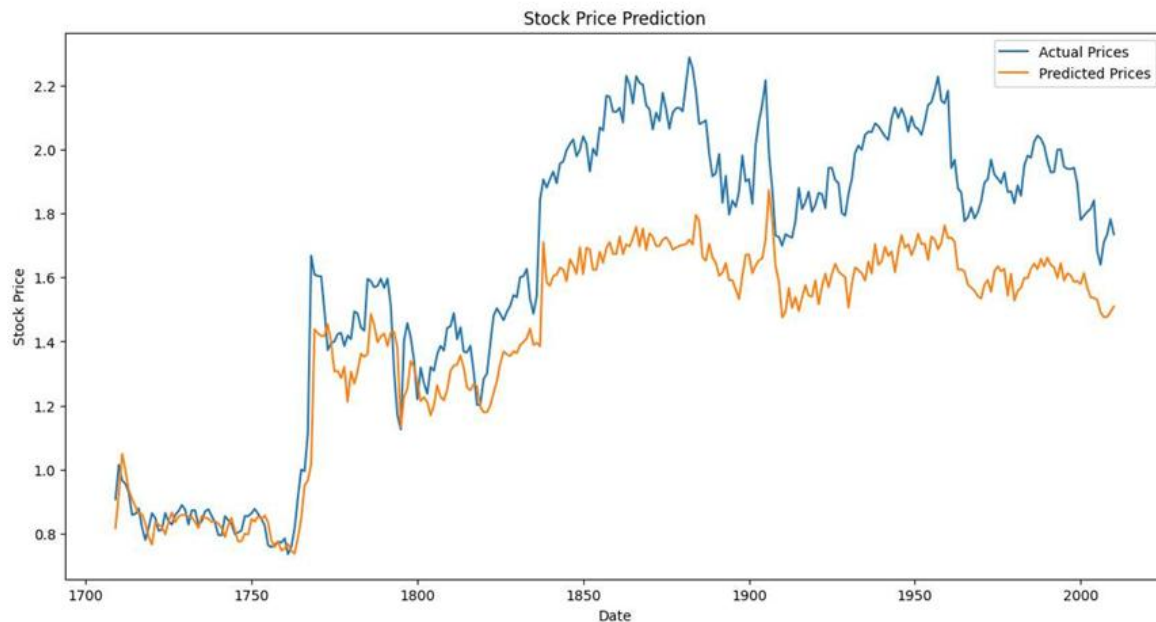
Layer (type)	Output Shape	Param #
lstm_24 (LSTM)	(None, 5, 100)	40,800
dropout_51 (Dropout)	(None, 5, 100)	0
lstm_25 (LSTM)	(None, 50)	30,200
dropout_52 (Dropout)	(None, 50)	0
dense_32 (Dense)	(None, 1)	51

Total params: 71,051 (277.54 KB)
Trainable params: 71,051 (277.54 KB)
Non-trainable params: 0 (0.00 B)



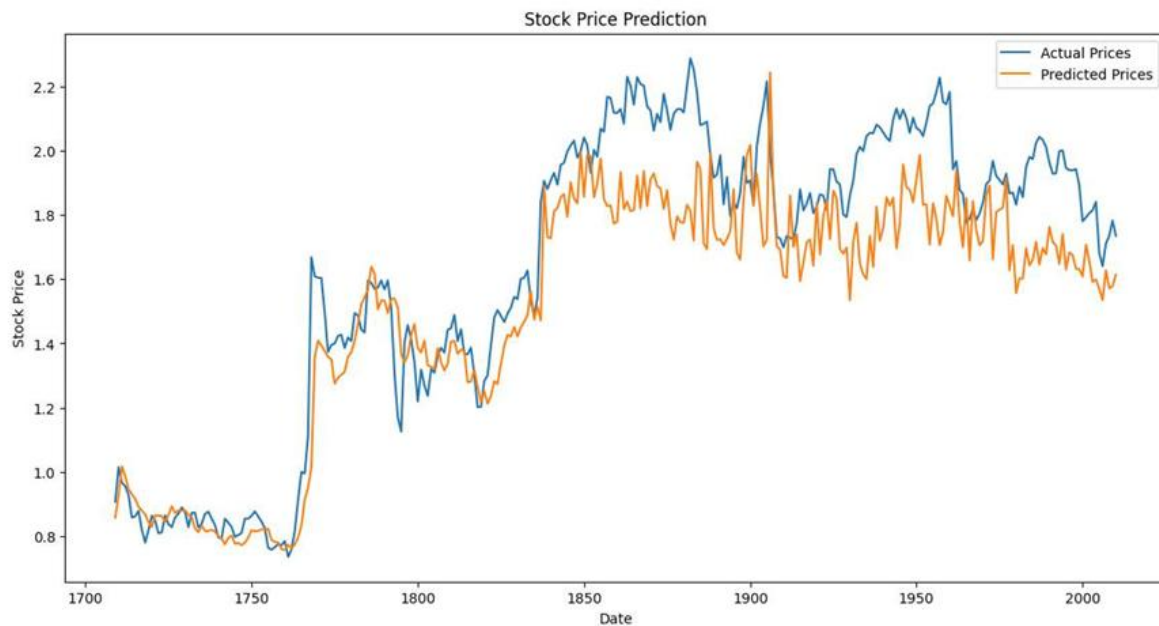
Mean Absolute Error (MAE): 0.238300276713504
Mean Squared Error (MSE): 0.0715262006017046
R-squared (R2): 0.6598030543549182

b. LSTM model with primary feature



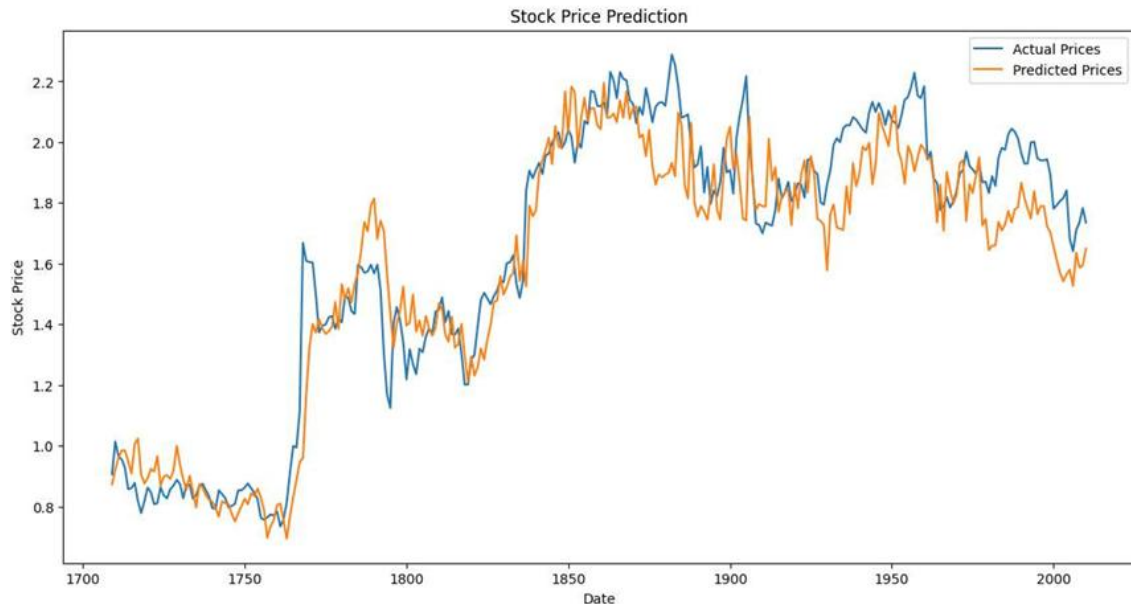
Mean Absolute Error (MAE): 0.2341345407217075
Mean Squared Error (MSE): 0.0781420758819417
R-squared (R2): 0.6283362555571106

c. LSTM model with additional features:



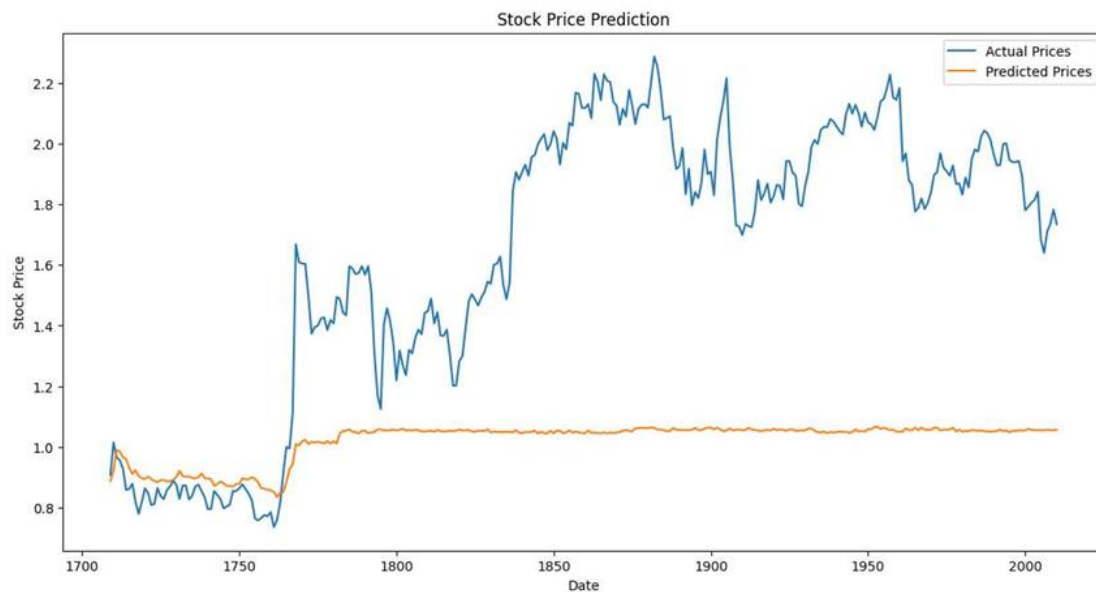
Mean Absolute Error (MAE): 0.15227778668535702
Mean Squared Error (MSE): 0.039438685968253874
R-squared (R2): 0.8124194995150383

d. LSTM model with all features:



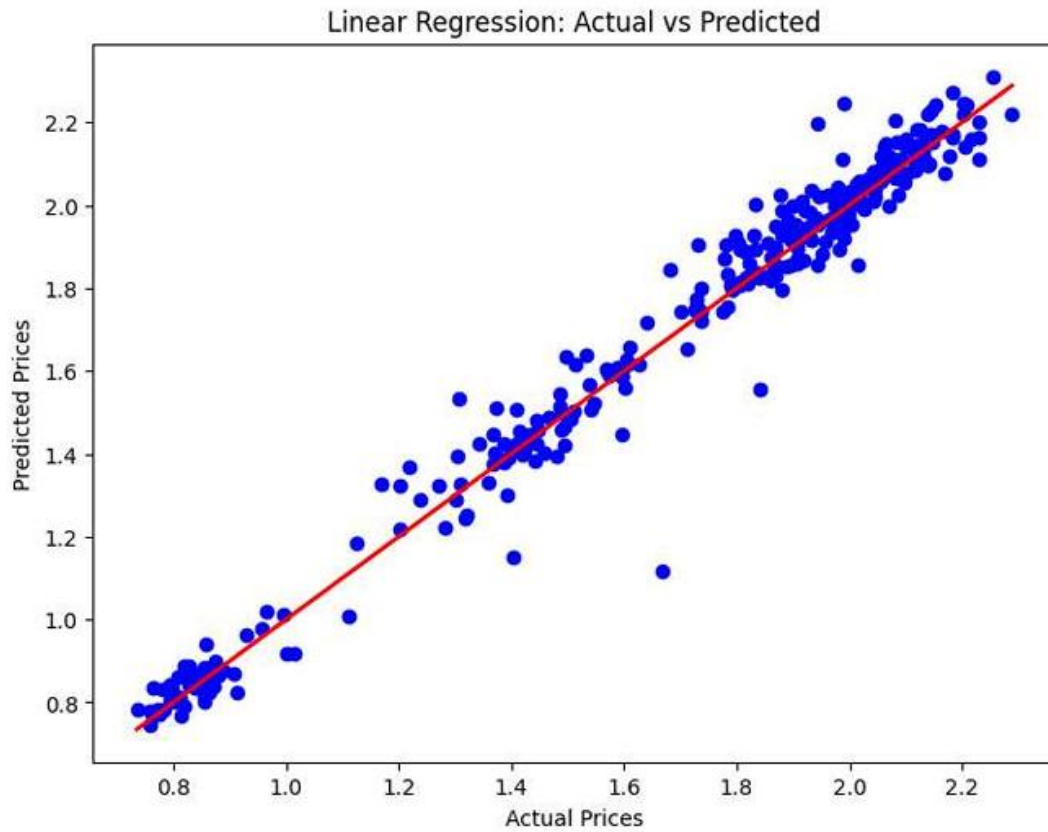
Mean Absolute Error (MAE): 0.1129249771037243
Mean Squared Error (MSE): 0.022586059230901963
R-squared (R2): 0.8925749124114858

e. Random Forest Model:



Tuned Random Forest - MAE: 0.631124603667435, MSE: 0.5379463355251942, R2: -1.5586106731114988

f. Linear Regressor:



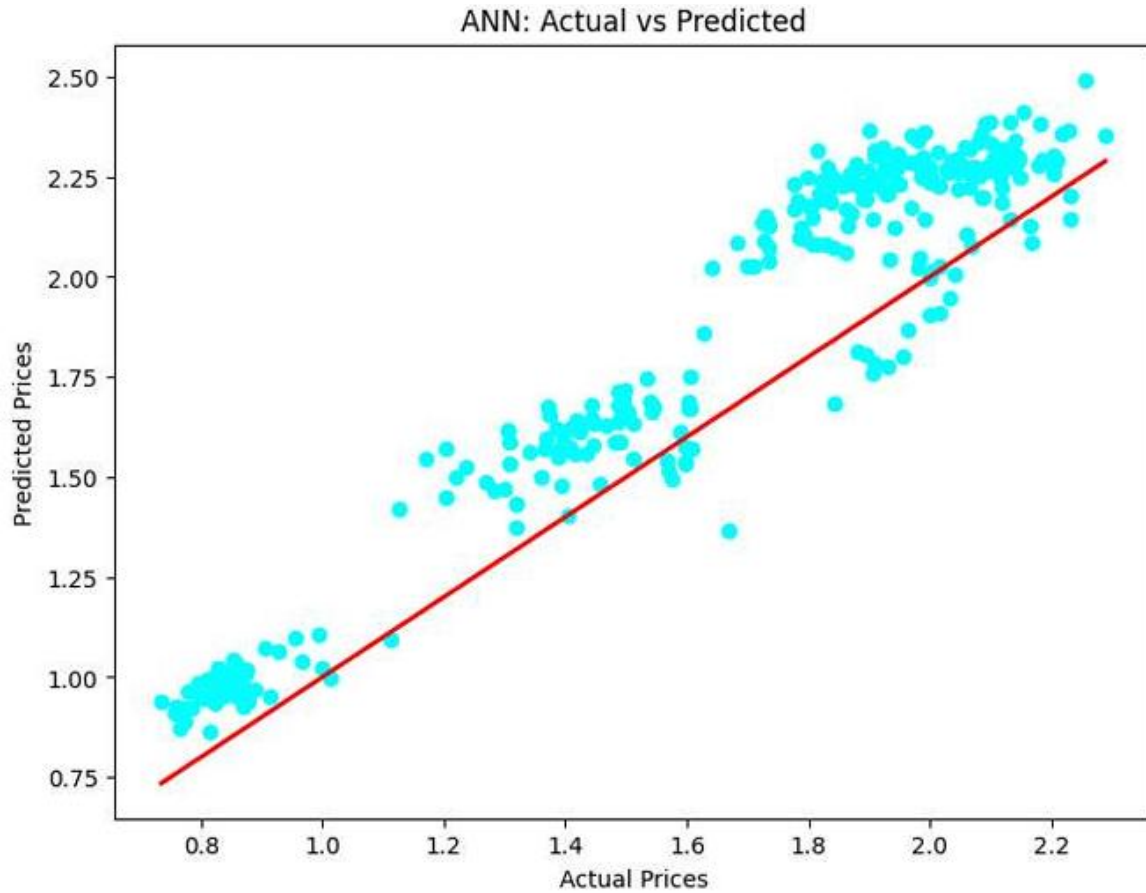
Linear Regression - MAE: 0.04793165238929389, MSE: 0.005155456640292655, R2: 0.9754793266288575

g. XgBoost Regressor:



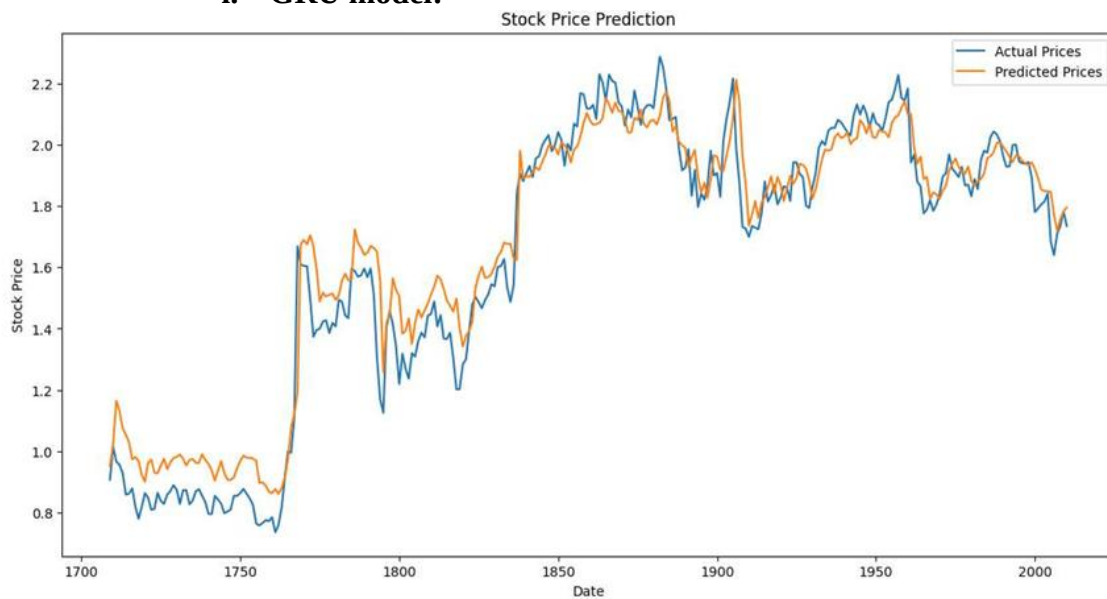
Tuned XGBoost - MAE: 0.605943026435084, MSE: 0.507464971693542, R2: -1.4136334928979388

h. ANN model:



ANN - MAE: 0.20441318912789294, MSE: 0.05447487068683572, R2: 0.7409035505007564

i. GRU model:



GRU Model - MAE: 0.08293228089474042, MSE: 0.011095913674122042, R2: 0.947224990152939

Summary of accuracy of different model:

Evaluation Type	LSTM model (without & with windowing)	GRU model(without & with windowing)
Using primary features	MAE: 0.2383 & 0.2020 MSE: 0.0715 & 0.0624 R2 score: 0.6598 & 0.6984	MAE: 0.088 & 0.1069 MSE: 0.01195 & 0.1930 R2 score: 0.9432 & 0.8166
Primary features with sentiment score	MAE: 0.2341 & 0.1170 MSE: 0.0781 & 0.0229 R2 score: 0.6283 & 0.8894	MAE: 0.1249 & 0.1002 MSE: 0.023 & 0.0197 R2 score: 0.8905 & 0.8126
With additional features	MAE: 0.1523 & 0.1835 MSE: 0.0394 & 0.0529 R2 score: 0.8124 & 0.7441	
With all extracted features	MAE: 0.1129 & 0.1594 MSE: 0.0226 & 0.0352 R2 score: 0.8926 & 0.8300	MAE: 0.0769 MSE: 0.0102 R2 score: 0.9515

Sentiment analysis score:

Positive sentiment:

Enter a sentence to analyze its sentiment: Googles free wifi at Indian railway stations is better than most of the countrys paid services

Sentiment score for 'Googles free wifi at Indian railway stations is better than most of the countrys paid services': 0.467

Enter a sentence to analyze its sentiment: Devastated: scientists too late to captive breed mammal lost to climate change - Australian conservationists spent 5 months obtaining permissions & planning for a captive breeding program. But when they arrived on the rodents tiny island, they they were too late.

Sentiment score for 'Devastated: scientists too late to captive breed mammal lost to climate change - Australian conservationists spent 5 months obtaining permissions & planning for a captive breeding program. But when they arrived on the rodents tiny island, they they were too late.': -0.043

Negative sentiment:

Enter a sentence to analyze its sentiment: US FTC issues warning to franchisors over unfair business practices

Sentiment score for 'US FTC issues warning to franchisors over unfair business practices': -0.500

5.1.2 Discussions:

After applying different models we find out that only LSTM and GRU model is giving better accuracy and better prediction. The reason of this because these two models have memory cell and they can forget any unnecessary texts, they can easily be used for forecasting related problem due to their memory cells. And that's why they give a better accuracy.

Now, between LSTM and GRU, the GRU model gives better accuracy than LSTM, the reasons of this are:

GRU has a simpler structure than LSTM. It uses only two gates — an update gate and a reset gate — compared to the three gates in LSTM. This simpler design allows GRU to train faster, require fewer parameters, and often generalize better, especially on smaller datasets or less complex sequences. In our case, since we are working with daily stock prices and sentiment scores, which may not require extremely long memory, the GRU model handled the temporal patterns more efficiently.

Another reason GRU might have outperformed LSTM is that LSTM is more prone to

overfitting if the dataset is not very large or noisy. GRU avoids this by keeping fewer gates and focusing on the most important patterns in the data. Also, the training process of GRU is generally more stable, which might have helped the model converge to a better solution with fewer epochs.

Lastly, stock market data often contains short to mid-term trends rather than long-term dependencies. GRU performs particularly well in such cases because it captures immediate changes and momentum better without trying to remember unnecessary long-term history, which LSTM might try to do.

On the other hand, the regressor models and ANN model fails to predict correctly because Traditional regression models (like Linear Regression or Random Forest Regressor) and simple feedforward neural networks often fail in stock price prediction because they do not understand time or sequence. They treat each data point as independent and cannot learn from previous days' trends, momentum, or patterns. Since stock prices are time-dependent, models that ignore the temporal structure end up giving poor or flat predictions. Also, these models struggle with the high noise and volatility in financial data, leading to underfitting or overly smoothed results that miss real market movements.

5.2 Limitations:

The main limitations we faced during this project is to find out the proper dataset. As our project is aimed to predict the closing stock market price which is also dependent on the sentiment score of some text data, so it very important to find out the proper datasets which are correlated each other.

But unfortunately, we didn't find these types of datasets which will match accurately. But we got almost close related datasets, for example the stock prize prediction dataset is for the tech company Google and we analysis sentiment for all technology related stock market. This was our main limitations.

6 Reflection on Individual and Team work (PO(i))

6.1 Individual Contribution of Each Member :

Student ID	Individual Contribution
2006137	Data collection and helps in building models for stock market prize prediction.
2006176	Helps in features extraction and building models for sentiment analysis.
2006186	Data collection and helps in determining the sentiment score.
2006187	Integrates sentiment scores with stock analysis dataset and helps in models.
2006190	Builds different types of RNN blocks and tried to increase the accuracy perfectly.
2006193	Performs hyper parameter tuning and built different regressor models for stock analysis.

6.2 Log Book of Project Implementation

Date	Timeline
04-06-2025	After assigning project by the course instructor, we all project mates meet with each other and discussed about the project and tried to find out the datasets but we could not find proper datasets.
17-06-2025	A meeting in the presence of each member was hold to set the road map for the project and finalized out desired datasets.
25-06-2025	We applied textBlob model on the text data to measure the sentiment score and integrate them with the stock market dataset.
29-06-2025	Additional features were extracted by the team members which could play an important role on better accuracy.
08-07-2025	LSTM is performed for different types of features and we observed the differences between various features extraction.
12-07-2025	Different regressor models are performed to prove that regressor does not work here properly.
16-07-2025	RNN based models and CNN based models are performed to compare between their accuracy.
22-07-2025	Hyper parameter tuning and callbacks are performed on different types of models to increase the accuracy.

28-07-2025	Final meeting with each member and prepare reports and discussed about the projects with each other.
-------------------	--

7 Communication to External Stakeholders (PO(j))

7.1 Executive Summary

We are excited to present our project on **Stock Market Price Prediction of Google using Sentiment Analysis**. By combining news headlines with machine learning, our system predicts future stock prices more intelligently. First, we analyze the emotional tone of news articles using a sentiment analysis tool, and then feed this information into a smart prediction model called GRU (Gated Recurrent Unit). This helps us understand how public mood influences the market. Our model has shown promising results and can support investors in making better decisions by blending news and data science in a modern, user-friendly way.

7.2 Github Link:

https://github.com/navin2006190/Stock_market_prediction_with_sentiment_analysis.git

8 Future Work (PO(l)):

Although our current model shows good performance by combining sentiment analysis with GRU-based prediction, there is still room for further improvement. In the future, we plan to include real-time news data and social media feeds such as tweets or Reddit discussions, which may provide more timely and diverse sentiment signals. Additionally, we aim to use more advanced sentiment models like BERT or RoBERTa, which understand deeper context and sarcasm better than TextBlob or VADER.

We also plan to improve the prediction model by experimenting with hybrid architectures that combine LSTM, GRU, and CNN layers for better feature extraction. Another direction is to apply attention mechanisms that allow the model to focus more on important time steps. Moreover, incorporating volume, technical indicators, and macroeconomic data can make the model more robust and realistic.

Finally, deploying the model as a web application or mobile app could help make it accessible for users and investors to get live predictions based on news sentiment and market trends.

9 References:

- I. <https://www.kaggle.com/datasets/aaron7sun/stocknews>
- II. https://www.researchgate.net/publication/339131017_Stock_Market_Prediction_Using_Long_Short-Term_Memory
- III. <https://arxiv.org/abs/2411.05790>
- IV. <https://ojs.sgsci.org/journals/iaet/article/download/162/152>
- V. https://miro.medium.com/v2/resize:fit:984/1*Mb_L_slY9rjMr8-IADHvvg.png
- VI. https://d2l.ai/_images/gru-3.svg
- VII. https://books.google.com/books?hl=en&lr=&id=QZ8DEQAAQBAJ&oi=fnd&pg=PA260&dq=stock+market+price+prediction+using+sentiment+analysis&ots=WV9xHAIVxq&sig=WM5pMAYh_-6f8eQz5BDhKw85Na8